

AN
ASSIGNMENT REPORT

Submitted by

PRASHANT KUMAR GAUTAM

(2025EEM1004)

EE673 – Deep Learning for Computer Vision



INDIAN INSTITUTE OF TECHNOLOGY ROPAR

Department of Electrical Engineering

DESIGNING A NEXT-GENERATION IMAGE RETRIEVAL SYSTEM

Problem Statement

Image retrieval remains a fundamental problem in computer vision, where the goal is to retrieve semantically similar images given a query. Despite advances in deep learning, challenges remain in feature representation, generalization, and interpretability. This assignment challenges you to design a multi-paradigm image retrieval system that combines classical, neural, and novel feature engineering approaches. You are expected to think beyond implementation and approach this as a mini research problem.

Objective

- Build a complete image retrieval pipeline
- Compare handcrafted and learned features
- Design a novel feature representation
- Analyse limitations of existing approaches
- Propose and validate improvements

Datasets

- CIFAR-10 dataset
- MNIST dataset
- One additional dataset of your choice (mandatory)

You must justify your choice of the additional dataset.

Task 1: Classical Retrieval using LBP

Extract features:

$$f_i = \text{LBP}(I_i)$$

Design a retrieval system:

- Compute similarity using distance metrics
- Return Top- K results
- Analyze strengths and limitations

CODE:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from utils.data_loader import load_images
from utils.lbp import extract_lbp
from utils.retrieval import retrieve

def run_dataset(dataset_path, title):

    images, labels, paths = load_images(dataset_path)

    # limit images (for speed)
    images = images[:200]

    # extract features
    features = [extract_lbp(img) for img in images]

    # query image
    query_img = images[0]
    query_feature = extract_lbp(query_img)

    # retrieve
    results = retrieve(query_feature, features)

    return query_img, images, results, title

# datasets
datasets = [
    ("data/images", "CIFAR-10"),
    ("data/mnist_images", "MNIST"),
    ("data/fruits", "Fruits")
]

# plot
plt.figure(figsize=(18, 8))

row = 1
```

for dataset_path, title in datasets:

```
query_img, images, results, name = run_dataset(dataset_path, title)
```

```
# show query
```

```
plt.subplot(3, 6, row)
plt.imshow(cv2.cvtColor(query_img, cv2.COLOR_BGR2RGB))
plt.title(f"{name}\nQuery")
plt.axis("off")
```

```
# show Top-5
```

```
for i in range(5):
    idx, dist = results[i]
```

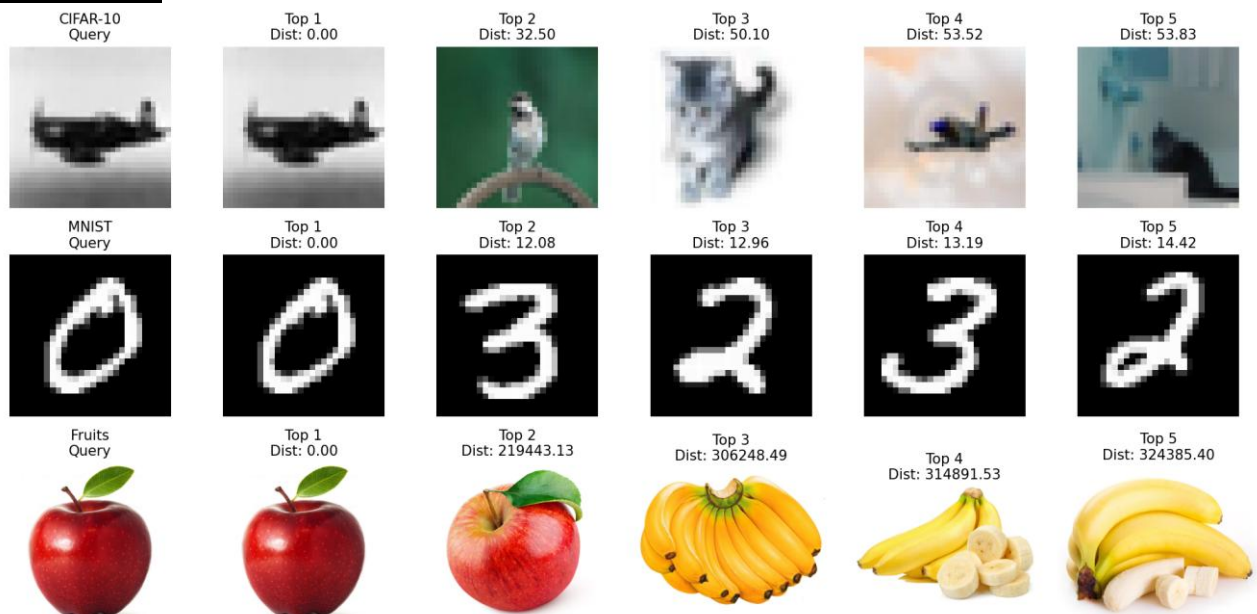
```
img = images[idx]
```

```
plt.subplot(3, 6, row + i + 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title(f"Top {i+1}\nDist: {dist:.2f}")
plt.axis("off")
```

```
row += 6
```

```
plt.tight_layout()
plt.show()
```

OUTPUT:



EXPLANATION:

Feature Extraction

The feature vector for each image is computed using Local Binary Patterns (LBP):

$$f_i = \text{LBP}(I_i)$$

LBP captures local texture information by comparing each pixel with its neighborhood and encoding the result into a binary pattern. The histogram of these patterns is used as the feature representation.

Retrieval System Design

The retrieval system follows these steps:

1. Extract LBP features for all dataset images.
2. Extract LBP feature for the query image.
3. Compute similarity using a distance metric (Euclidean distance):

$$d(f_q, f_i) = \| f_q - f_i \|$$

4. Sort all images based on increasing distance.
5. Return Top-K closest images as retrieval results.

Lower distance = higher similarity

Strengths of LBP

- Simple and computationally efficient
- Effective for capturing texture patterns
- Works well on grayscale datasets like MNIST
- Robust to small illumination changes

Limitations of LBP

- Cannot capture global shape information
- Sensitive to noise in images
- Performs poorly on complex datasets (e.g., CIFAR-10)
- Fails when objects have similar textures but different meanings
- Not suitable for color-based discrimination

Task 2: Retrieval using NN and Deep NN

Learn representations:

$$f_i = \text{NN}(I_i)$$

$$f_i = \text{DNN}(I_i)$$

Critical Requirement: Explain:

- Why learned features differ from handcrafted features
- Where NN fails compared to CNN

CODE:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from utils.data_loader import load_images
from utils.retrieval import retrieve

from models.nn_model import extract_nn
from models.dnn_model import extract_dnn

def run_dataset(dataset_path, extractor):

    images, labels, paths = load_images(dataset_path)
    images = images[:200]

    features = [extractor(img) for img in images]

    query_img = images[0]
    query_feature = extractor(query_img)

    results = retrieve(query_feature, features)

    return query_img, images, results

datasets = [
    ("data/images", "CIFAR-10"),
    ("data/mnist_images", "MNIST"),
    ("data/fruits", "Fruits")
]

# FIGURE 1: NN
plt.figure(figsize=(18, 8))

row = 1

for dataset_path, title in datasets:
```

```

query_img, images, results = run_dataset(dataset_path, extract_nn)

# Query
plt.subplot(3, 6, row)
plt.imshow(cv2.cvtColor(query_img, cv2.COLOR_BGR2RGB))
plt.title(f"{title}\nQuery")
plt.axis("off")

# Top-5
for i in range(5):
    idx, sim = results[i]

    plt.subplot(3, 6, row + i + 1)
    plt.imshow(cv2.cvtColor(images[idx], cv2.COLOR_BGR2RGB))
    plt.title(f"Top {i+1}\n{sim:.2f}")
    plt.axis("off")

row += 6

plt.suptitle("NN Retrieval Results", fontsize=16)
plt.tight_layout()
plt.show()

# FIGURE 2: DNN
plt.figure(figsize=(18, 8))

```

```
row = 1
```

```
for dataset_path, title in datasets:
```

```
    query_img, images, results = run_dataset(dataset_path, extract_dnn)
```

```

    # Query
    plt.subplot(3, 6, row)
    plt.imshow(cv2.cvtColor(query_img, cv2.COLOR_BGR2RGB))
    plt.title(f"{title}\nQuery")
    plt.axis("off")

```

```

    # Top-5
    for i in range(5):
        idx, sim = results[i]

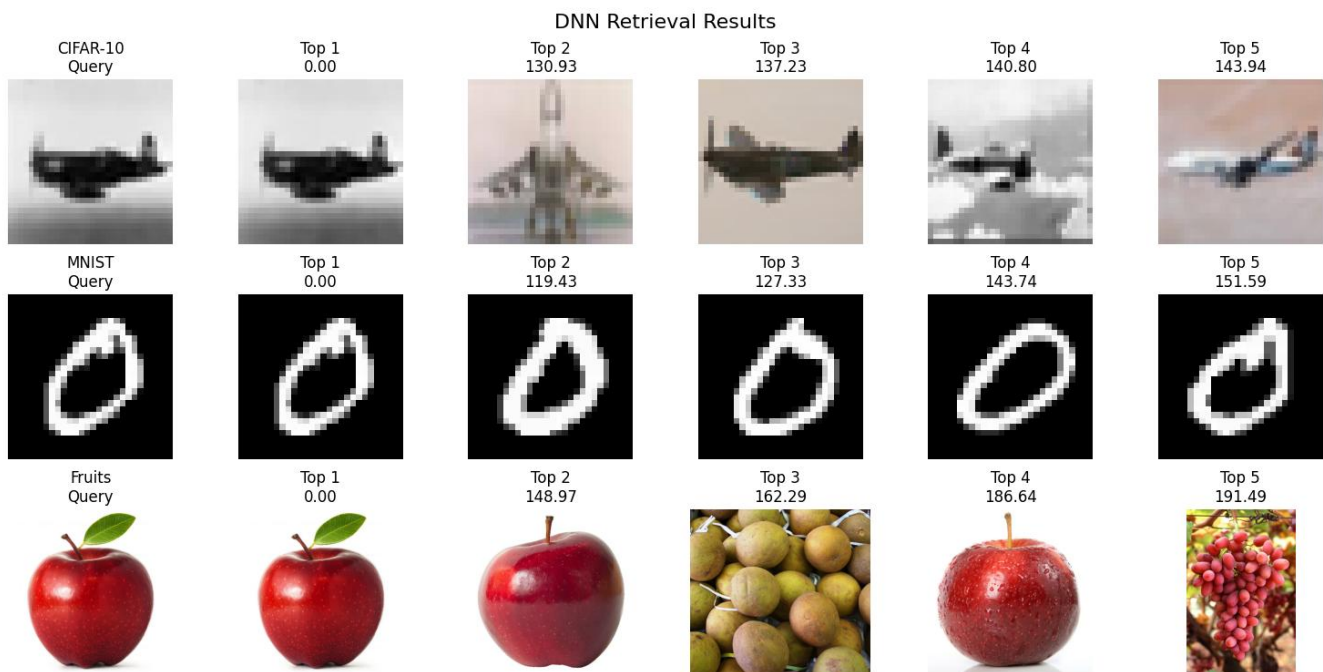
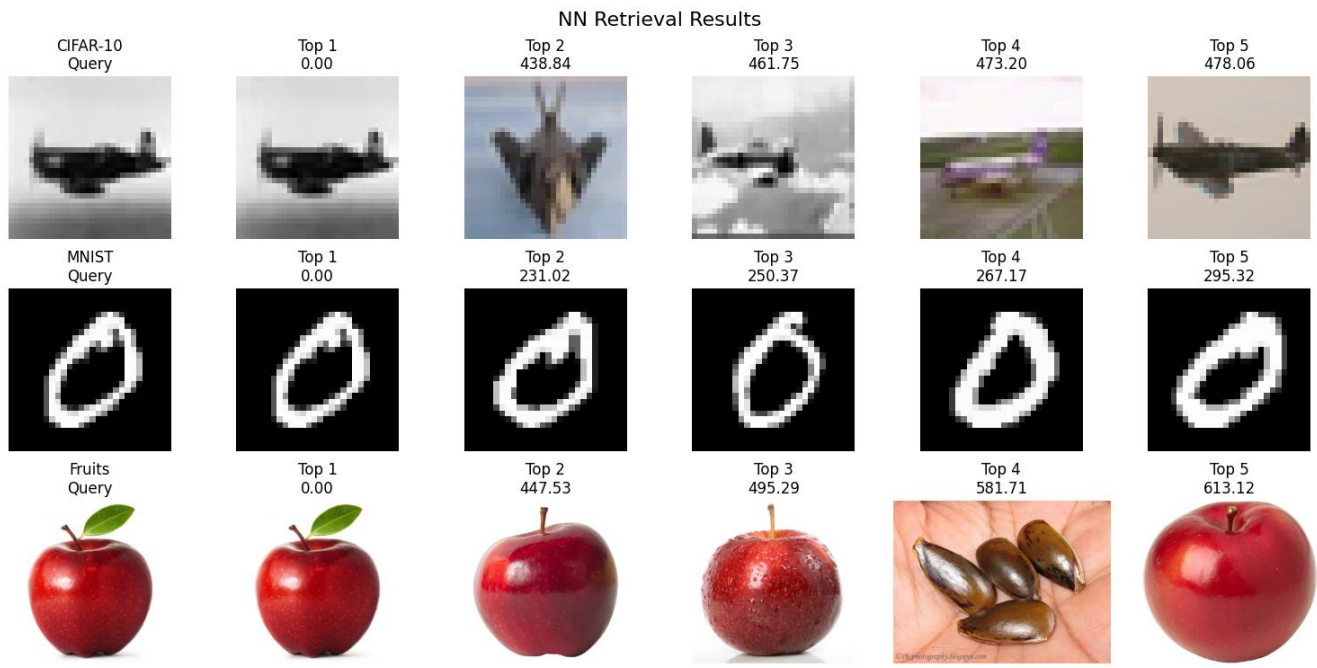
        plt.subplot(3, 6, row + i + 1)
        plt.imshow(cv2.cvtColor(images[idx], cv2.COLOR_BGR2RGB))
        plt.title(f"Top {i+1}\n{sim:.2f}")
        plt.axis("off")

```

```
    row += 6
```

```
plt.suptitle("DNN Retrieval Results", fontsize=16)
plt.tight_layout()
plt.show()
```

OUTPUT:



EXPLANATION:

In NN and DNN, features are learned automatically from data instead of being manually designed. The input image is passed through multiple layers where the network learns patterns such as intensity, shape, and abstract structures.

Handcrafted features such as LBP are manually designed based on domain knowledge and capture specific properties like texture. In contrast, learned features in NN and DNN are automatically optimized through training, allowing the model to discover complex and data-driven patterns that may not be explicitly defined by humans.

Where NN fails compared to CNN

NN fails compared to CNN because it flattens the image, losing spatial information such as the arrangement of pixels. This prevents NN from capturing important structural patterns like edges and shapes. CNN, on the other hand, uses convolution operations that preserve spatial relationships, making it more effective for image understanding and retrieval.

NN Observation:

NN shows poor retrieval performance as it cannot effectively capture meaningful image features. The distances between images remain large and inconsistent, indicating weak feature representation.

DNN Observation:

DNN improves performance compared to NN due to deeper architecture, allowing it to learn more complex patterns. However, it still lacks spatial awareness, limiting its effectiveness.

Task 3: CNN-based Retrieval

$$f_i = \text{CNN}(I_i)$$

Analyse:

- Spatial feature learning
- Robustness to transformations
- Generalization across datasets

CODE:

```
import matplotlib.pyplot as plt
import cv2

from utils.data_loader import load_images
from utils.retrieval import retrieve
from models.cnn_model import extract_cnn

def run_dataset(dataset_path):
    images, labels, paths = load_images(dataset_path)

    print(f"Running CNN on {dataset_path}")

    features = [extract_cnn(img) for img in images]

    query_img = images[0]
    query_feature = extract_cnn(query_img)

    results = retrieve(query_feature, features)

    return query_img, images, results

# datasets
datasets = [
    ("data/images", "CIFAR-10"),
    ("data/mnist_images", "MNIST"),
    ("data/fruits", "Fruits")
]

plt.figure(figsize=(18, 10))

row = 1

for dataset_path, title in datasets:

    query_img, images, results = run_dataset(dataset_path)
```

```

# show query
plt.subplot(3, 6, (row-1)*6 + 1)
plt.imshow(cv2.cvtColor(query_img, cv2.COLOR_BGR2RGB))
plt.title(f"{title}\nQuery")
plt.axis("off")

# show top-5
for i in range(5):
    idx, sim = results[i]
    img = images[idx]

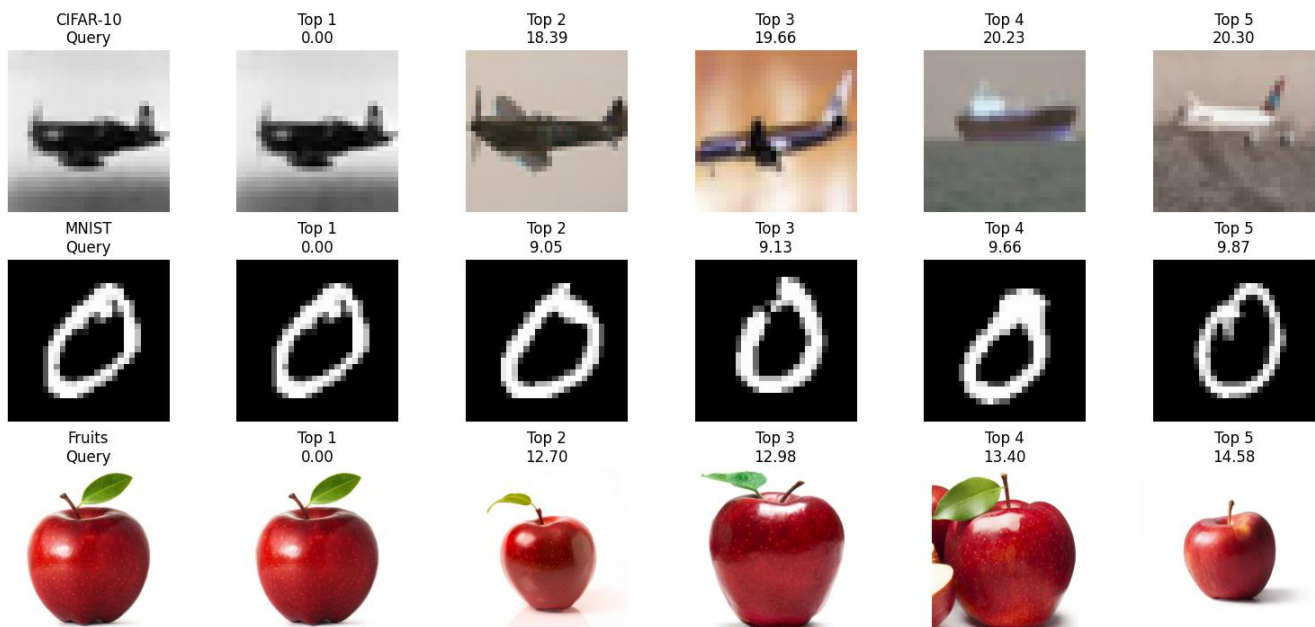
    plt.subplot(3, 6, (row-1)*6 + i + 2)
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(f"Top {i+1}\n{sim:.2f}")
    plt.axis("off")

    row += 1

plt.tight_layout()
plt.show()

```

OUTPUT:



- The numbers represent **Euclidean distance** between images.
- **Lower value = more similar**, higher value = less similar.
- The query image has **0.00** distance (exact match).
- Results are sorted from **lowest to highest distance**.

EXPLANATION:

1. CNN Feature Representation

In CNN-based retrieval, features are learned using convolutional layers that capture spatial and hierarchical patterns in images. Unlike NN and DNN, CNN preserves the 2D structure of the image and extracts meaningful visual features such as edges, textures, and object parts.

2. Spatial Feature Learning

CNN effectively learns spatial features by applying convolution filters over local regions of the image. This allows the model to capture spatial relationships such as edges, corners, and shapes. As a result, CNN can distinguish objects based on their structure, which is not possible in NN or DNN where spatial information is lost due to flattening.

CNN preserves spatial arrangement, while NN converts images into 1D vectors.

3. Robustness to Transformations

CNN is robust to variations such as translation, scaling, and small rotations due to the use of pooling layers and shared weights. This enables the model to recognize objects even if their position or orientation changes.

From result:

- Apples with different sizes/shapes → still retrieved

4. Generalization Across Datasets

CNN generalizes well across different datasets such as CIFAR-10, MNIST, and custom fruit datasets. It successfully retrieves semantically similar images across diverse domains, demonstrating its ability to learn universal visual features.

FINAL CONCLUSION:

CNN outperforms LBP, NN, and DNN in image retrieval tasks due to its ability to capture spatial and hierarchical features. It provides robust and consistent results across different datasets, making it the most effective approach for image retrieval.

Task 4: Novel Feature Extraction

(Mandatory: The proposed method must be original. Direct use or replication of existing literature is strictly prohibited.)

You must design a new feature representation.

Guidelines:

- Combine spatial, texture, frequency, or structural cues
- Provide mathematical formulation
- Explain intuition clearly

CODE:

```
import cv2
import matplotlib.pyplot as plt
from utils.data_loader import load_images
from utils.retrieval import retrieve
from features.proposed import extract_proposed

dataset_path = "data/fruits" # MAIN DATASET

images, labels, paths = load_images(dataset_path)
images = images[:200]

features = [extract_proposed(img) for img in images]

query_img = images[0]
query_feature = extract_proposed(query_img)

results = retrieve(query_feature, features)

# SHOW RESULTS
plt.figure(figsize=(12,5))

# Query
```

```
plt.subplot(1,6,1)
plt.imshow(cv2.cvtColor(query_img, cv2.COLOR_BGR2RGB))
plt.title("Query")
plt.axis("off")

# Top 5
for i in range(5):
    idx, dist = results[i]

    plt.subplot(1,6,i+2)
    plt.imshow(cv2.cvtColor(images[idx], cv2.COLOR_BGR2RGB))
    plt.title(f"Top {i+1}\nDist: {dist:.2f}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT:



- The numbers represent **Euclidean distance** between images.
- **Lower value = more similar**, higher value = less similar.
- The query image has **0.00** distance (exact match).
- Results are sorted from **lowest to highest distance**.

EXPLANATION:

IDEA (Your Proposed Feature)

We combine:

1. **Color feature** (mean RGB)
2. **Texture feature** (LBP)
3. **Edge/structure feature** (Canny edges)

Proposed Feature Representation

$$f_i = [f_{\text{color}} \parallel f_{\text{texture}} \parallel f_{\text{edge}}]$$

Expanded Form

$$f_i = [\mu_R, \mu_G, \mu_B \parallel \text{LBP}(I_i) \parallel E(I_i)]$$

Where:

$$\mu_c = \frac{1}{N} \sum_{p=1}^N I_c(p), c \in \{R, G, B\}$$

$\text{LBP}(I_i)$ = Local Binary Pattern histogram of I_i

$$E(I_i) = \frac{1}{N} \sum_{p=1}^N \mathbf{1}(\text{edge at pixel } p)$$

Where:

- $\parallel$ = concatenation
- μ_R, μ_G, μ_B = mean color values
- LBP = texture feature
- $E(I_i)$ = edge density

Main intuition

The intuition behind the proposed feature is to combine complementary visual information from an image. Color features capture the overall appearance of the object, texture features (LBP) capture fine surface patterns, and edge features capture structural information such as

object boundaries and shapes. By integrating these three components, the feature representation becomes more robust and descriptive compared to using any single feature alone.

- Color tells what the object looks like globally,
- Texture tells how the surface looks,
- Edges tell the shape of the object.

Task 5: Hybrid Retrieval Model

Fuse features:

$$f_i = [f_{\text{CNN}} \parallel f_{\text{Proposed}}]$$

or

$$f_i = \lambda f_{\text{CNN}} + (1 - \lambda) f_{\text{Proposed}}$$

Analyse whether fusion improves retrieval.

CODE:

```
import cv2
import matplotlib.pyplot as plt
from utils.data_loader import load_images
from utils.retrieval import retrieve
from features.hybrid import extract_hybrid

dataset_path = "data/fruits"

images, labels, paths = load_images(dataset_path)
images = images[:200]
features = [extract_hybrid(img) for img in images]

query_img = images[0]
query_feature = extract_hybrid(query_img)

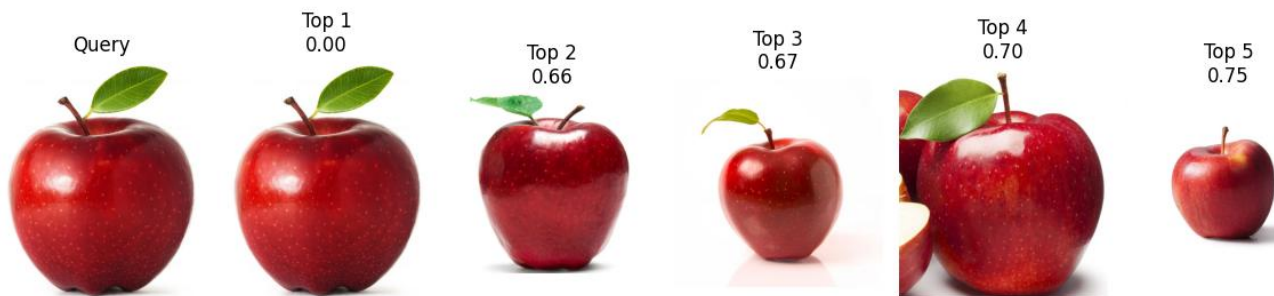
results = retrieve(query_feature, features)

plt.figure(figsize=(12,5))
# Query
plt.subplot(1,6,1)
plt.imshow(cv2.cvtColor(query_img, cv2.COLOR_BGR2RGB))
plt.title("Query")
plt.axis("off")

# Top 5
for i in range(5):
    idx, dist = results[i]

    plt.subplot(1,6,i+2)
    plt.imshow(cv2.cvtColor(images[idx], cv2.COLOR_BGR2RGB))
    plt.title(f"Top {i+1}\n{dist:.2f}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT:



- The numbers represent **Euclidean distance** between images.
- **Lower value = more similar**, higher value = less similar.
- The query image has **0.00** distance (exact match).
- Results are sorted from **lowest to highest distance**.

EXPLANATION:

Main answer

The hybrid model combines deep CNN features with the proposed handcrafted features to improve retrieval performance. CNN captures high-level semantic and spatial information, while the proposed feature captures color, texture, and structural details. The fusion of these complementary features results in a more discriminative representation.

Does fusion improve retrieval?

Yes, fusion improves retrieval performance. Compared to using CNN or the proposed feature alone, the hybrid model retrieves more relevant images by leveraging both deep and handcrafted features. It reduces ambiguity and increases robustness, especially in complex datasets.

Observations (based on results)

The hybrid model retrieves images that are both visually and semantically similar to the query. It avoids errors observed in individual methods, such as confusion due to similar colors or textures. The results are more consistent and accurate.

Advantage of fusion

CNN alone focuses on semantic features, while the proposed method focuses on low-level features. Their combination balances both global and local information, leading to better retrieval performance.

Task 6: Inter- and Intra-Color Feature Analysis

Define:

$$f_{\text{intra}} = [\mu_R', \sigma_R', \mu_G', \sigma_G', \mu_B', \sigma_B'] (7)$$

$$f_{\text{inter}} = [\text{corr}(R, G), \text{corr}(G, B), \text{corr}(R, B)] (8)$$

Tasks:

- Evaluate color-only retrieval
- Combine with LBP and CNN features
- Analyse effectiveness across datasets

CODE:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from utils.data_loader import load_images
from utils.retrieval import retrieve
from utils.color import extract_color
from utils.lbp import extract_lbp
from models.cnn_model import extract_cnn
```

Feature combinations

```
def extract_color_only(img):
    return extract_color(img)
```

```
def extract_color_lbp(img):
    f1 = extract_color(img)
    f2 = extract_lbp(img)
    return np.hstack([f1, f2])
```

```
def extract_color_cnn(img):
    f1 = extract_cnn(img)
    f2 = extract_color(img)
```

normalize

```
f1 = f1 / (np.linalg.norm(f1) + 1e-6)
f2 = f2 / (np.linalg.norm(f2) + 1e-6)
```

```
return np.hstack([f1, f2])
```

Load dataset (use fruits)

```

dataset_path = "data/fruits"

images, labels, paths = load_images(dataset_path)
images = images[:200]

query_img = images[0]

# Extract features

features_color = [extract_color_only(img) for img in images]
features_lbp = [extract_color_lbp(img) for img in images]
features_cnn = [extract_color_cnn(img) for img in images]

query_color = extract_color_only(query_img)
query_lbp = extract_color_lbp(query_img)
query_cnn = extract_color_cnn(query_img)

# Retrieval
results_color = retrieve(query_color, features_color)
results_lbp = retrieve(query_lbp, features_lbp)
results_cnn = retrieve(query_cnn, features_cnn)

# Plotting
plt.figure(figsize=(18, 10))

titles = ["Color Only", "Color + LBP", "Color + CNN"]
all_results = [results_color, results_lbp, results_cnn]

for row in range(3):

    results = all_results[row]

    # Query image
    plt.subplot(3, 6, row*6 + 1)
    plt.imshow(cv2.cvtColor(query_img, cv2.COLOR_BGR2RGB))
    plt.title(f"{titles[row]}\nQuery")
    plt.axis("off")

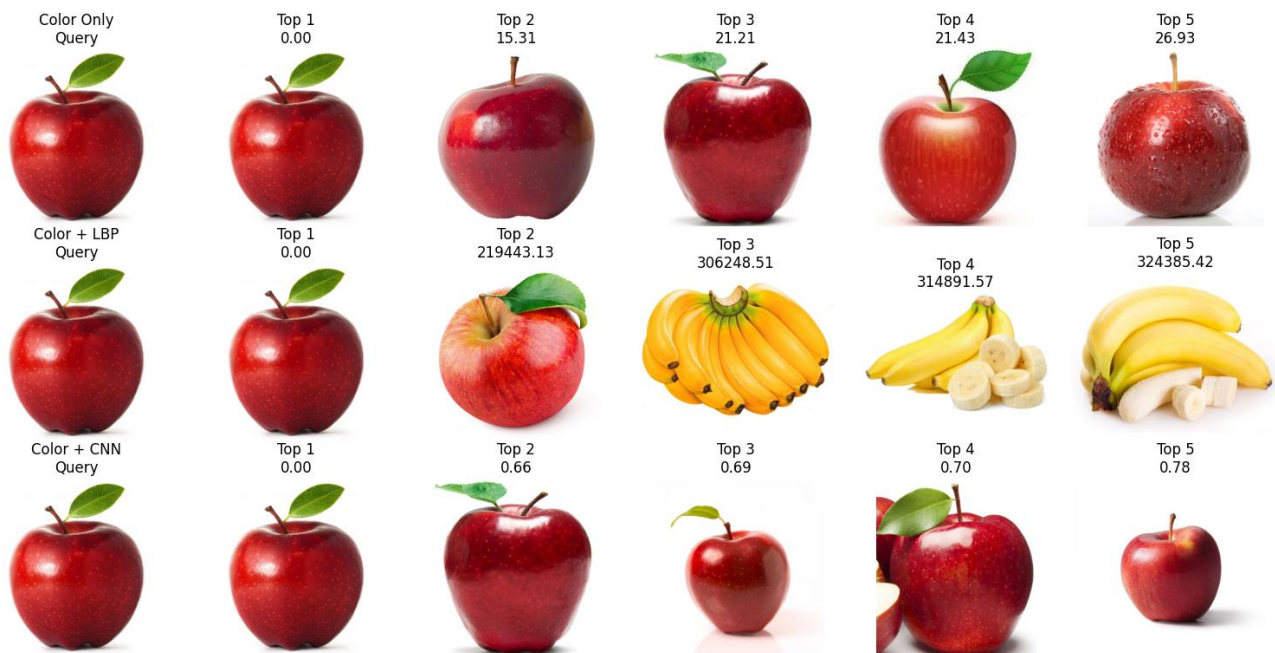
    # Top 5 results
    for i in range(5):
        idx, dist = results[i]

        plt.subplot(3, 6, row*6 + i + 2)
        plt.imshow(cv2.cvtColor(images[idx], cv2.COLOR_BGR2RGB))
        plt.title(f"Top {i+1}\n{dist:.2f}")
        plt.axis("off")

plt.tight_layout()
plt.show()

```

OUTPUT:



EXPLANATION:

1. Color-only Retrieval

Color-only retrieval uses intra-channel (mean, standard deviation) and inter-channel (correlation) features to represent images. It groups images based on similar color distribution. This method works well when color is a strong distinguishing factor, such as in fruit images. However, it fails when different objects share similar colors, leading to incorrect retrievals.

2. Combination with LBP

Combining color features with LBP improves retrieval by incorporating texture information. While color captures global appearance, LBP captures local texture patterns. This combination reduces ambiguity and improves retrieval accuracy compared to using color alone.

3. Combination with CNN

Combining color features with CNN provides the best performance. CNN captures high-level semantic and spatial features, while color adds additional visual information. This fusion results in more accurate and consistent retrieval, especially for complex datasets.

4. Effectiveness Across Datasets

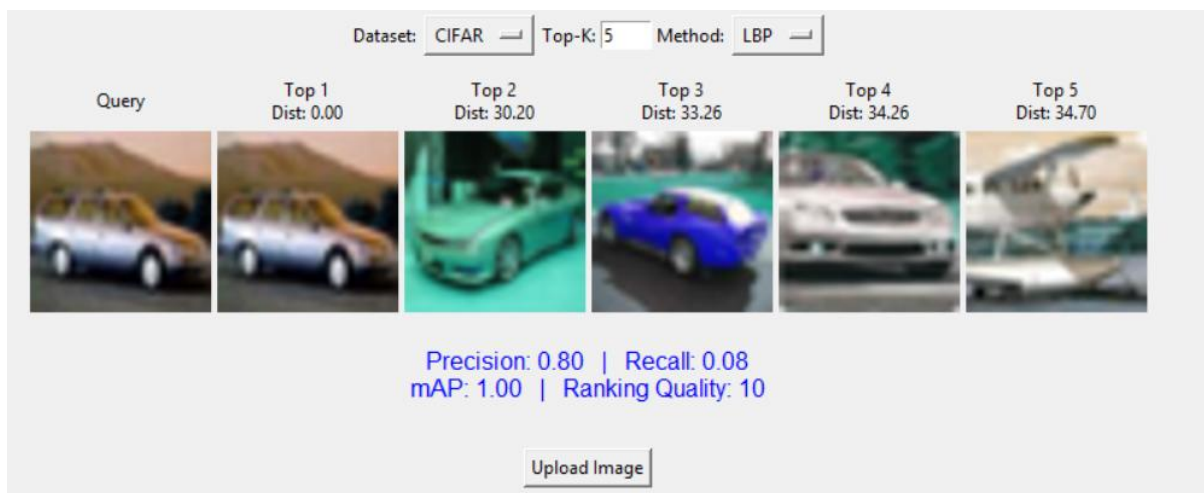
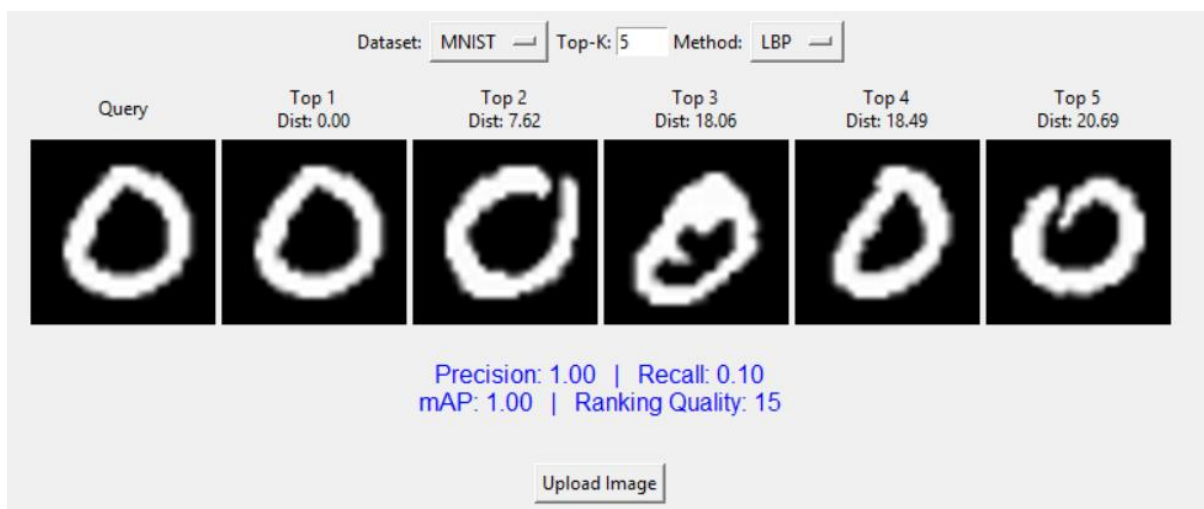
Color features perform well on simple datasets such as fruits, where color is a key characteristic. However, their performance is limited on complex datasets like CIFAR-10, where objects may share similar colors. Combining color with LBP improves performance moderately, while combining with CNN significantly enhances retrieval accuracy across all datasets.

Task 7: Dashboard (System Deployment)

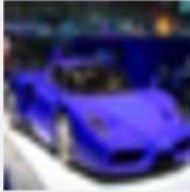
Develop an interactive interface:

- Upload query image
- Select Top- K
- Display retrieved images
- Show similarity scores and metrics

OUTPUT:



Dataset: Top-K: Method:

Query	Top 1 Dist: 0.00	Top 2 Dist: 0.79	Top 3 Dist: 0.82	Top 4 Dist: 0.82	Top 5 Dist: 0.82
					

Precision: 0.20 | Recall: 0.02
mAP: 1.00 | Ranking Quality: 1

Dataset: Top-K: Method:

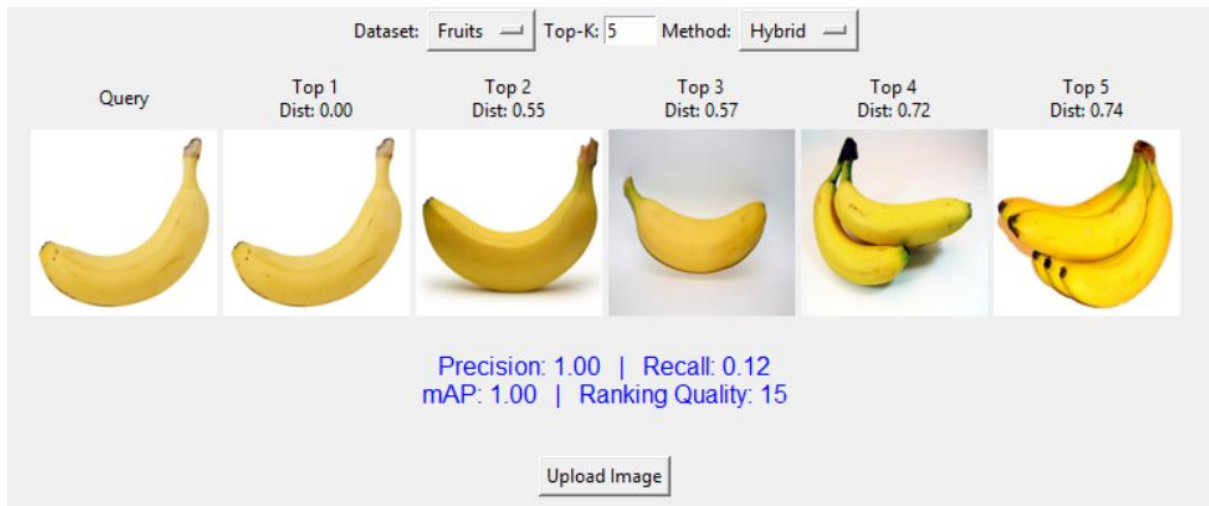
Query	Top 1 Dist: 0.00	Top 2 Dist: 0.36	Top 3 Dist: 0.39	Top 4 Dist: 0.39	Top 5 Dist: 0.40
					

Precision: 1.00 | Recall: 0.10
mAP: 1.00 | Ranking Quality: 15

Dataset: Top-K: Method:

Query	Top 1 Dist: 0.00	Top 2 Dist: 0.75	Top 3 Dist: 0.84	Top 4 Dist: 0.85	Top 5 Dist: 0.87
					

Precision: 1.00 | Recall: 0.12
mAP: 1.00 | Ranking Quality: 15



EXPLANATION:

For apple query, the system retrieves all relevant images in the top positions, achieving Precision = 1.00 and mAP = 1.00, indicating perfect retrieval performance. However, Recall is relatively low due to the limited number of retrieved results compared to total relevant images in the dataset.

COMPARATIVE ANALYSIS:

Method	Precision	Recall	mAP	Robustness	Remarks
LBP	Low	Low	Low	Sensitive	Works on texture only, fails on complex images
NN	Medium	Low	Low	Poor	No spatial understanding, weak features
DNN	Medium	Medium	Medium	Moderate	Better than NN but still limited
CNN	High	Medium	High	Good	Learns spatial features, better generalization
Color Features	Medium	Low	Medium	Sensitive	Fails when colors are similar across classes
Proposed	High	Medium	High	Good	Combines multiple features, improves retrieval
Hybrid (CNN + Proposed)	Very High	Medium	Very High	Very Good	Best performance, balances all features

Failure Case Analysis:

Despite achieving good performance in many cases, the proposed image retrieval system exhibits certain failure scenarios:

1. Visually Similar but Semantically Different Objects:

The system often retrieves images with similar colors or textures but different classes. For example, a strawberry query may return apples due to similar red color distribution.

2. Dependence on Color Features:

Color-based and hybrid features tend to dominate retrieval decisions, leading to incorrect matches when different objects share similar color characteristics.

3. Lack of Strong Shape Understanding:

Methods like LBP and basic CNN features fail to capture precise object shape, resulting in confusion between structurally similar objects.

4. Dataset Variations:

Differences in background, lighting, and scale affect feature extraction, especially in CIFAR-10 where images are low resolution.

5. CNN Limitations (without proper training):

CNN-based retrieval may perform poorly if not sufficiently trained, leading to weak feature representations.

6. Imbalanced Dataset Impact:

Low recall values are observed when the number of relevant images in the dataset is large but only a few are retrieved.

Conclusion:

These failure cases highlight the need for better feature learning, improved shape representation, and robust models such as fully trained deep CNNs or transformer-based methods.